32 x 32 Ternary Content Addressable Memory (TCAM)

ECE 4740 Final Project

Md Shad (mss464), Kaelem Bent (kab472), Albert Sun (ays48), Carolyn Kavanagh (cek227)

Abstract

In this project, we designed and implemented a high-speed 32 × 32-bit Ternary Content Addressable Memory (TCAM) using several VLSI design techniques learned throughout the semester. Unlike traditional memory like SRAMS, which retrieves data from a specific address, a TCAM searches all stored entries in parallel and returns a match in a single operation. Our design includes several major subcircuits, including TCAM bitcells, dynamic matchline precharge and evaluation circuitry, input key generation cells, and a 32-to-5 priority encoder that outputs the address of the first matching entry from highest to lowest. Each bitcell stores both the true and complementary values of a bit, allowing the cell to represent logic '0', logic '1', and a "don't care" state required for ternary searching. Throughout the design process, we focused on optimizing speed, power, and layout area while maintaining reliable operation during both write and search phases. Overall, this project demonstrates an efficient and scalable TCAM architecture capable of fast parallel searching while applying several key VLSI concepts covered throughout the course, including dynamic and domino-style logic, hierarchical memory array design, transistor sizing and logical effort optimization, parasitic-aware layout extraction, RC delay analysis, matchline precharge/evaluation techniques, power-delay tradeoff optimization, and high-speed digital memory design.

Introduction

As computer networks and modern digital systems continue to increase in speed, there is a growing need for hardware that can search large amounts of data extremely quickly. Traditional memory systems, such as SRAM and DRAM, retrieve data by first receiving an address and then accessing the data stored at that location. While this works well for general storage, it becomes inefficient when the system must search through many entries to find a match. Ternary content-addressable memories (TCAMs) solve this problem by allowing all stored entries to be compared against incoming search data at the same time. Instead of checking one location after another, a TCAM performs a fully parallel search and immediately returns the matching result, making search operations significantly faster than conventional memory access.

TCAMs are especially important in routers and network switches, where billions of packets may need to be processed every second. When a packet arrives at a router, the router must quickly determine where that packet should be forwarded based on its destination IP address. However, routers usually do not search for one exact address. Instead, they use longest-prefix matching, where only part of the address needs to match a routing rule. This is where the ternary functionality of a TCAM becomes extremely useful. In addition to storing logic '0' and logic '1', a TCAM can also store a "don't care" state, meaning that specific bits can be ignored during a search. For example, a routing rule may only require the first several bits of an IP address to match while the remaining bits can be either '0' or '1'. The "don't care" state allows TCAMs to efficiently perform these prefix-based searches in parallel, enabling routers and switches to make forwarding decisions in a single cycle. Because of this capability, TCAMs are widely used in packet forwarding, routing tables, firewall filtering, packet classification, and other high-speed search applications. Our motivation for the project was to apply the VLSI design concepts learned throughout the semester to a realistic high-performance digital system. Designing this TCAM also brought challenges not commonly seen in traditional static memory systems, such as

dynamic matchline operation, parallel comparison across an entire array, and precise timing between precharge, search, and evaluation phases.

Theory of Operation and Specifications

The 32 × 32 TCAM consists of 32 rows, where each row stores a 32-bit ternary word capable of fully parallel search operations. During a search, an incoming 32-bit search key is compared against every stored word simultaneously. Each row contains a matchline (ML) that indicates whether the stored word matches the incoming search data. Unlike conventional memories that access data sequentially through an address decoder, the TCAM performs all comparisons in parallel and determines matching entries within a single evaluation cycle. Each TCAM bitcell stores both the true and complementary values of a bit, allowing the cell to represent logic '0', logic '1', and a ternary "don't care" state. The "don't care" state is especially important in networking applications because routers commonly perform longest-prefix matching rather than exact address matching. A stored "don't care" allows a bit position to match either a '0' or '1' during the search operation.

The match condition for each row can be represented as:

$$\mathbf{ML(i) = \Pi\ Match(i,j)\ for\ j = 0\ to\ 31}$$

where Match(i,j) represents the comparison result of bit j in row i. A row's matchline remains high only if all 32 bit comparisons evaluate as matches. If any bit mismatches the search key, the corresponding bitcell creates a discharge path that pulls the matchline low.During operation, all matchlines are first dynamically precharged high. The search key is then broadcast across the search lines (SL) and complementary search lines (SLB). Each bitcell evaluates the incoming search data, and rows that fully match remain high while mismatched rows discharge low. The outputs of all matchlines are then passed into a 32-to-5 priority encoder, which outputs the binary address of the highest-priority matching row.

The search delay of the TCAM is primarily determined by the RC delay of the matchline and the delay through the priority encoder:

$$\mathbf{tML \propto RML \times CML}$$

As the number of rows and columns increases, matchline capacitance, search line loading, and routing parasitics also increase, resulting in larger delay and higher dynamic power consumption. Dynamic power can be approximated by:

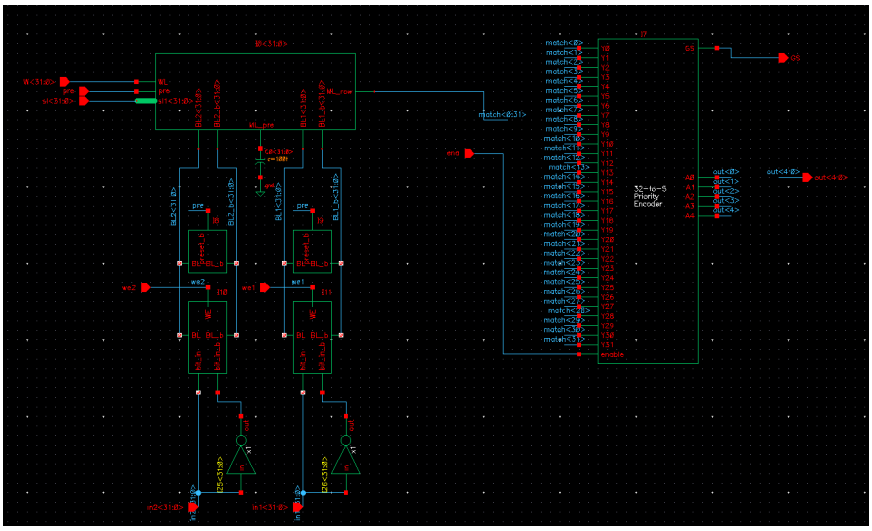
$$\mathbf{P_{dynamic} = \alpha C V_{DD}^2 f}$$

where α is the switching activity factor, C is the switched capacitance, VDD is the supply voltage, and f is the operating frequency.

To improve search speed, dynamic precharge and evaluation circuitry was used instead of fully static logic, reducing delay through the critical search path. However, dynamic logic introduces

additional challenges such as charge sharing, leakage sensitivity, and strict timing requirements between precharge and evaluation phases. Careful transistor sizing, buffering, and layout organization were therefore required to balance speed, power consumption, signal integrity, and overall scalability.

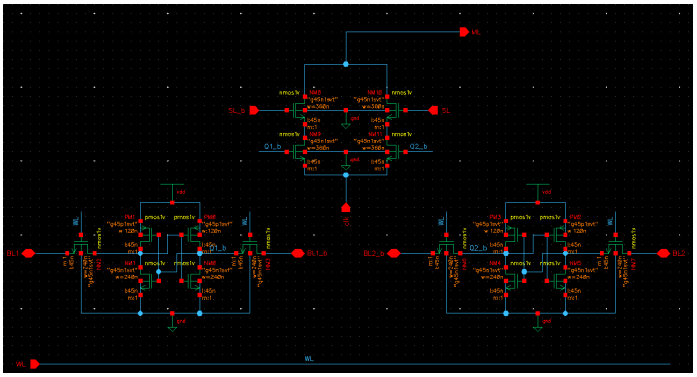
Top Level Diagram

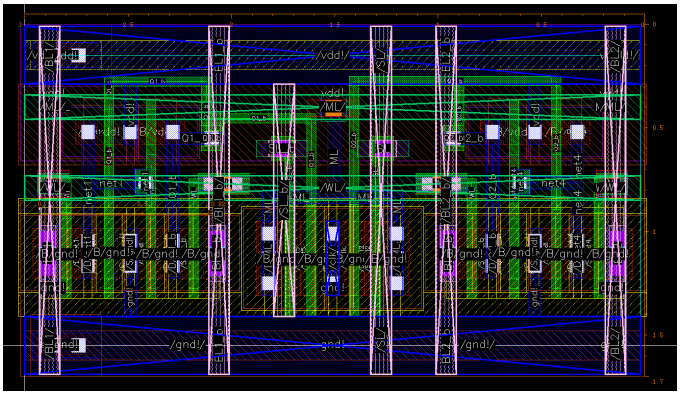


The top-level 32 × 32 TCAM is organized as an array of 32 rows, where each row stores a 32-bit ternary word and performs parallel search operations independently. The design consists of several main sub-blocks, including the TCAM bitcell array, input key generation circuitry, dynamic matchline precharge and evaluation circuitry, write control logic, and a 32-to-5 priority encoder. Together, these blocks allow the TCAM to compare incoming search data against all stored entries simultaneously and output the address of the highest-priority matching row. Incoming search data is converted into true and complementary search signals and broadcast across the array using search lines (SL) and complementary search lines (SLB). Before evaluation, all matchlines are dynamically precharged high. During the search phase, each bitcell compares the stored value against the incoming search key. Any mismatch discharges the corresponding matchline low, while fully matched rows remain high. The outputs of all matchlines are then passed into the priority encoder, which converts the active matchline into a 5-bit binary output address.

Sub-cell Circuits

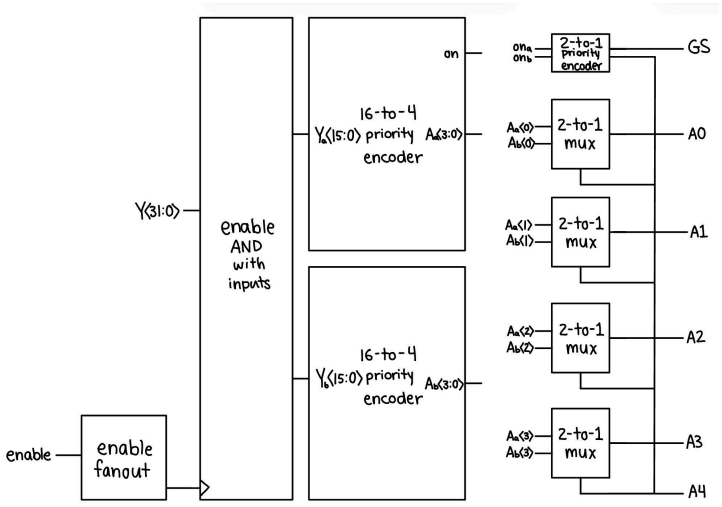
TCAM Bitcell:





The TCAM bitcell stores one ternary bit using two internal storage nodes, allowing it to represent logic ‘0’, logic ‘1’, or the “don’t care” state X. During a search operation, the stored data is compared against the incoming search signals SL and SLB. A stored ‘1’ matches when SL = 1, a stored ‘0’ matches when SL = 0, and a stored X matches either search value. If a mismatch occurs, the bitcell creates a discharge path that pulls the matchline low. The bitcell was carefully optimized to balance reliable storage, fast matchline evaluation, and low parasitic loading while minimizing layout area. To make layout easier, we made sure that the bit lines, select lines, wordlines and matchlines run across the cell for tiling.

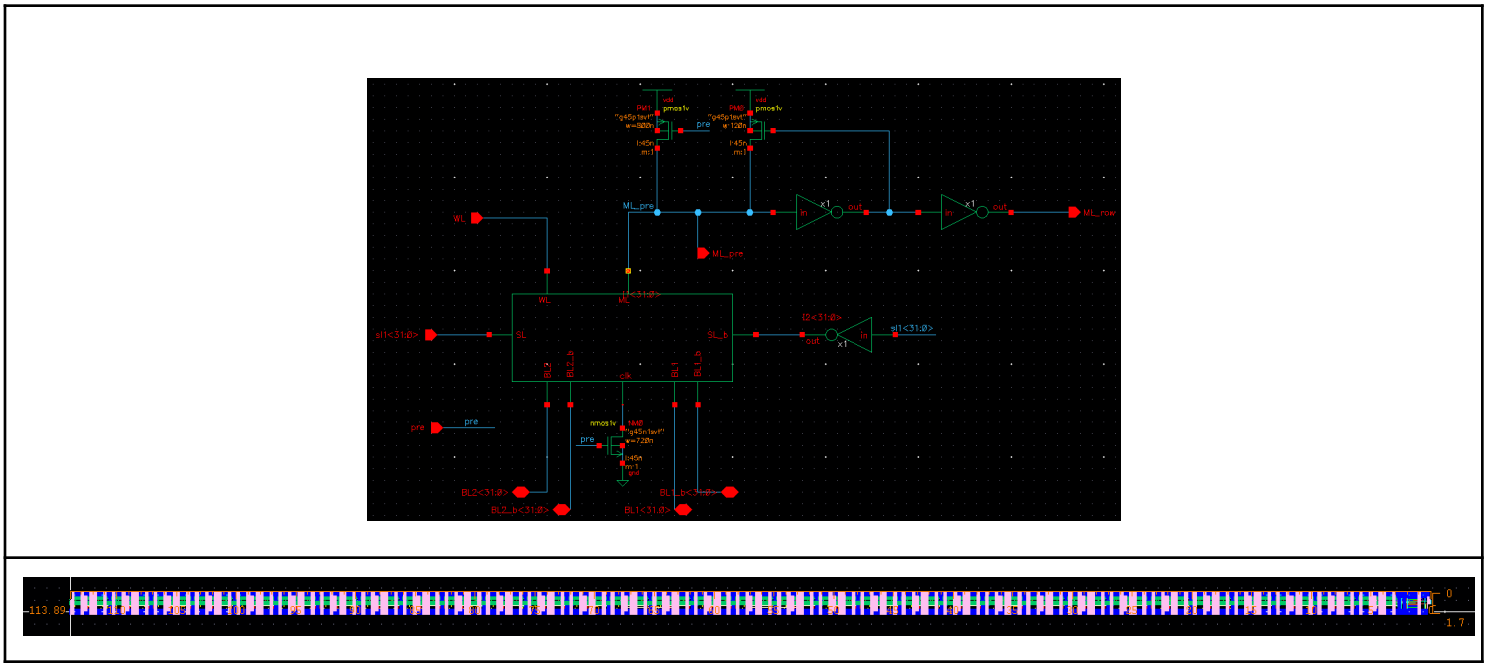
32 to 5 Priority Encoder:



The 32-to-5 priority encoder returns the 5-bit address of the highest priority input and asserts the group select (GS) output to indicate that a valid match was detected. A priority encoder is needed as opposed to a regular encoder because multiple match lines can be on at the same time if the incoming data matches with multiple rows. Since speed is a priority, the encoder was implemented using parallel comparison logic to reduce the length of the critical path and minimize output latency.

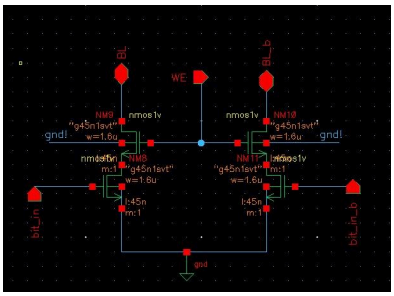
The 32 inputs are first compared in pairs by a layer of 2-to-1 priority encoders. This subcell returns a 0 for the A output if only input Y0 is on and a 1 if Y1 is on (even if Y0 is also on). It also has an on output to indicate if at least one of the two is on. The on outputs of two adjacent 2-to-1 priority encoders are fed into another 2-to-1 priority encoder and the A output of that encoder is used as the select signal of a 2-to-1 mux to choose which of the A outputs of the first two encoders is higher priority. This becomes the A0 of the highest priority address corresponding to a group of four inputs. The A output of the second priority encoder also becomes the A1 output of this group of inputs. This process is repeated with an increasing number of 2-to-1 muxes as the number of bits needed to store the highest priority address increases. The A output of the 2-to-1 encoders drives the select signals of more and more muxes, so the drive strength of the 2-to-1 encoders of the later layers is larger because a 4x inverter is used to drive the output. This ensures that the maximum fanout anywhere in the circuit is 3. The layout of the subcells was designed to be the same height as the bitcell, so that the matchlines of the memory array line up with the inputs of the encoder, and to make power routing easier. The enable input is routed such that the signal will arrive at higher priority inputs faster to prevent glitching.

TCAM Row:

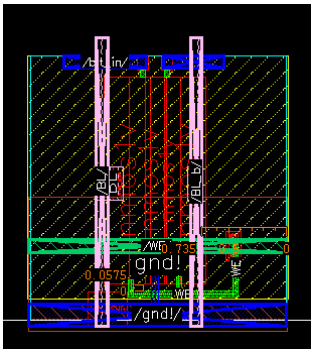


Each TCAM row consists of 32 bitcells connected to a shared dynamic matchline, allowing the row to store a complete 32-bit ternary word. Before evaluation, the matchline is precharged high. During the search phase, every bitcell in the row simultaneously compares its stored value against the incoming search key. The matchline remains high only if all 32 bits match or are stored as X. A mismatch at any bit position immediately discharges the matchline low, causing the entire row to evaluate as a mismatch. In case of a match, we use an inverter to feedback into the output to provide a stable signal. Since we need both SL and SL_b, we decided to generate SL_b for each column. This allowed us to have much easier routing in layout since we only needed to deal with one select line. The row architecture was designed to minimize matchline capacitance and RC delay in order to improve overall search speed.

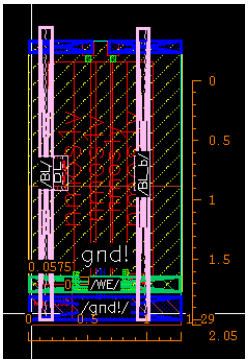
Column Write Circuit:



Column Write Layout

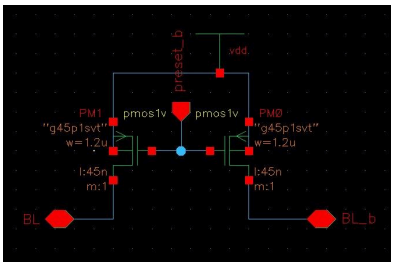


Alternate Column Write Layout

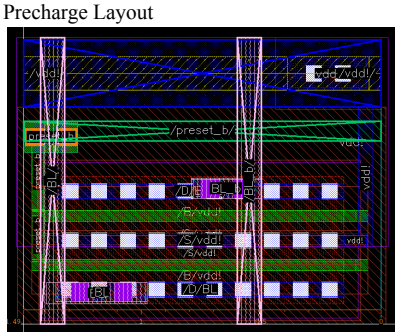


The column write circuit controls writing data into the TCAM array by driving the complementary bitlines BL and BLB during the write phase. When the write enable (WE) signal is active, the selected bitcell stores the incoming data through the bitlines. When WE is disabled, the bitlines are electrically isolated to prevent unintended writes and reduce unnecessary switching activity. Transistor sizing and buffering were optimized to provide sufficient write strength while minimizing power consumption and bitline loading across the array. One of our key considerations for layout was we wanted to minimize the amount of free space. We made 2 versions of the column write cells to fill up the entire column width. This allowed us to speed up the layout process significantly as all the components would tile well. While we could have combined all of the transistors into one sub-block and performed routing to save space, this made the layout much cleaner.

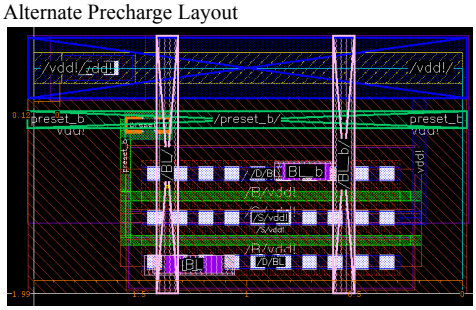
Precharge Circuit:



Precharge Layout

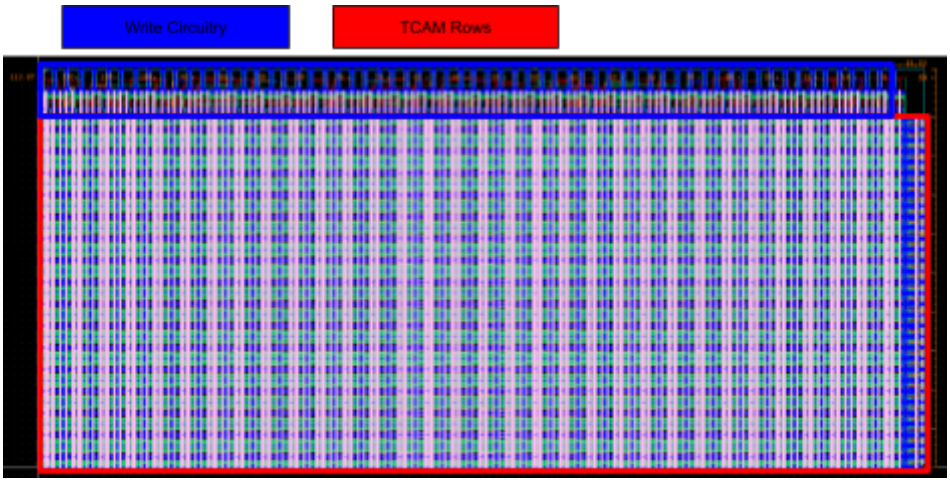


Alternate Precharge Layout



The precharge circuit dynamically charges the matchlines and complementary bitlines to VDD before each search or write operation begins. During the precharge phase, PMOS precharge transistors pull the lines high to establish the initial evaluation condition. Once precharge is disabled, the lines are released and allowed to evaluate dynamically during the search phase. The precharge circuitry was optimized to achieve fast charging while minimizing leakage, charge sharing, and unnecessary dynamic power consumption. Similar to the write cell, we created two alternate versions of the layout in order to fill out the column making tiling much easier.

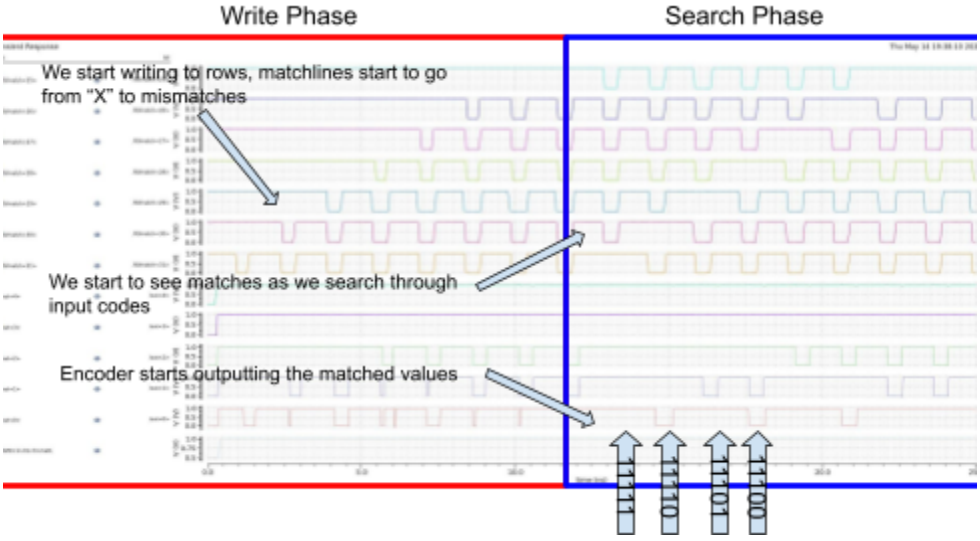
Top Level layout:



The top level layout consists of 32 row subcells stacked on top of each other. Each column is driven by two sets of write logic since each TCAM bitcell consists of 2 individual storage cells.

Simulation Results:

Schematic simulation results for the **write and search phases** are shown below. During the write phase, the encoder output is not yet valid, resulting in temporary glitches in the waveform. This behavior is expected due to simultaneous storage-node updates and matchline evaluation timing. Once the write operation completes, the outputs settle correctly and the subsequent search/read operation produces stable encoded results. The plot below shows 8 write cycles and 8 read cycles of unique codes.

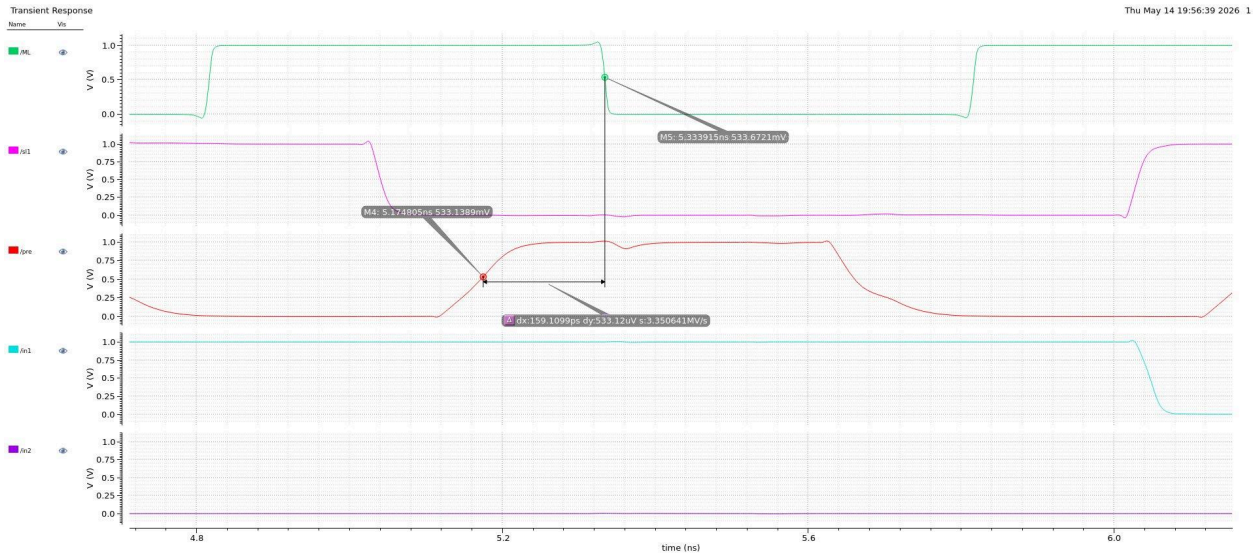


Clock period: 1.5 ns
Search latency: 900 ps

Write latency: 300 ps
Energy over 8 write operations: 12.4 pJ
Energy over 8 read operations: 13.4 pJ
Priority encoder output rise time: 12.7 ps
Priority encoder output fall time: 13.1 ps

Bitcell Extracted Single-Pulse Search - Single search pulse simulation of the extracted TCAM bitcell showing matchline evaluation timing after precharge. The measured search delay is taken from the completion of precharge, when the search key becomes valid, to the resulting ML response.

Search time: 159.1 ps
Matchline rise time: 9.0 ps
Matchline fall time: 7.8 ps



Array Extracted Write and Search

Due to LVS issues, we were unable to extract the full top-level, so we extracted each individual subcell. We believe the issue has to do with how Cadence handles hierarchy since we see the same nets across both layout and schematic however LVS named them differently. Tracing these nets we say that they were physically the same. We attempted to just lay out only row cells and redid the layout of the top-level several times however still encountered the same issue.

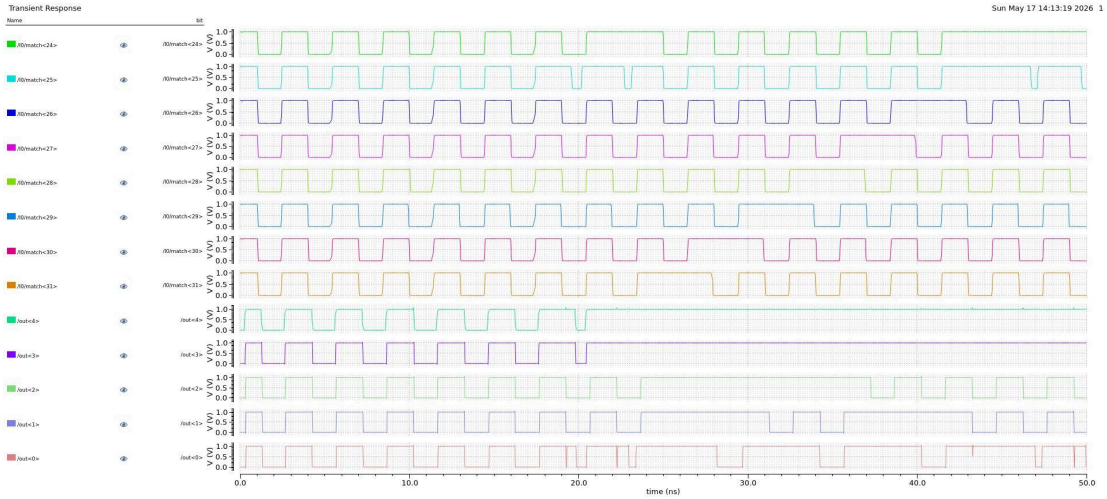
Schematic FET instances

```
                                                          |
                                                          | SerMos2#1
** missing connection **                                  |    B: XI0<0>/XI1<24>/MNM10  MN(G45N1SVT)
** missing connection **   . . . . . . . . . . . . . .   |    B: XI0<0>/XI1<24>/MNM11  MN(G45N1SVT)
                                                          |
```

Layout FET instances

```
Terminal: Layout Instance                                  | Terminal: Schematic Instance
-----------------------------------------------------------+-------------------------------
SerMos2#1                                                  |
   B: X678/X1/X1/X468/X48/M0 @(67.795,43.640)  MN(g45n1svt)| ** missing connection **
   B: X678/X1/X1/X468/M0 @(67.690,43.640)  MN(g45n1svt)    | ** missing connection **
   S: X678/X1/X1/X468/M0 @(67.690,43.640)  MN(g45n1svt)  . . .| ** missing connection **
```

Due to our use of dynamic logic, the extracted simulation increased our overall capacitance which affected our timings from schematic. This caused us to increase our precharge times as we found that our match and select lines were unable to pull down fast enough with the increased parasitics. With the increased time we can see that we have logically the same behavior as the schematic in the search phase. Interestingly, in the write phase we see that we only have mismatch behavior. This is likely because the bitcells started as “always 0” instead of “don’t care”



Clock period: 3 ns
Search latency: 1.1ns
Write latency: 600ps
Energy over 8 write operations: 52.3pJ
Energy over 8 read operations: 53.4pJ

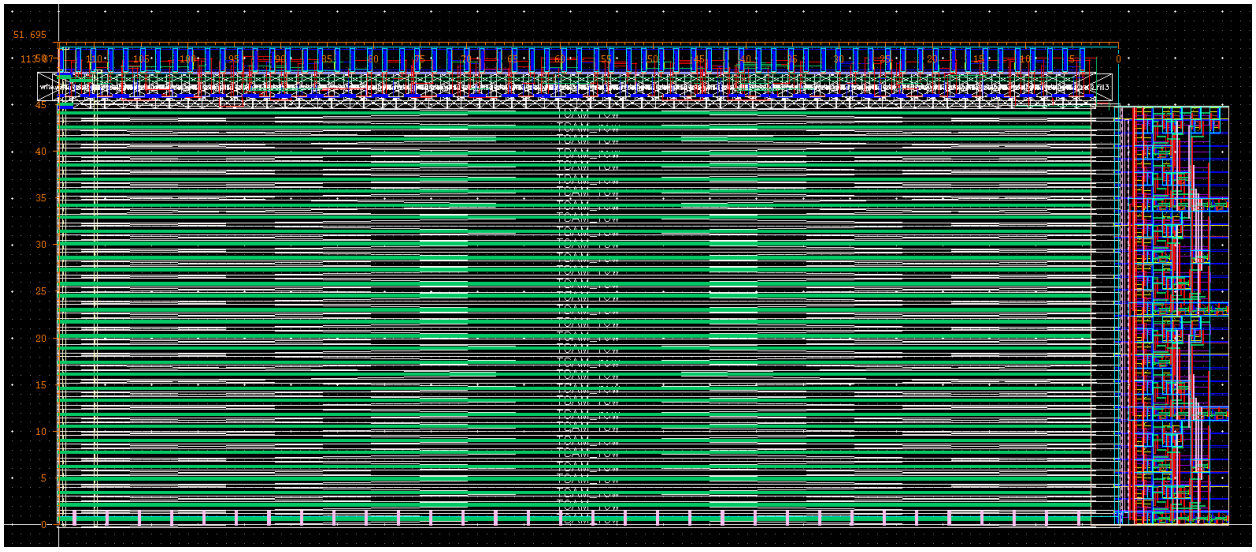
Discussion:

One of the most challenging aspects of the design was achieving reliable timing coordination between the precharge, search, and evaluation phases of TCAM operation. Since the matchlines rely on dynamic behavior, even small timing mismatches could produce incorrect matches, incomplete evaluations, or unintended discharge events. Developing accurate testbenches for the

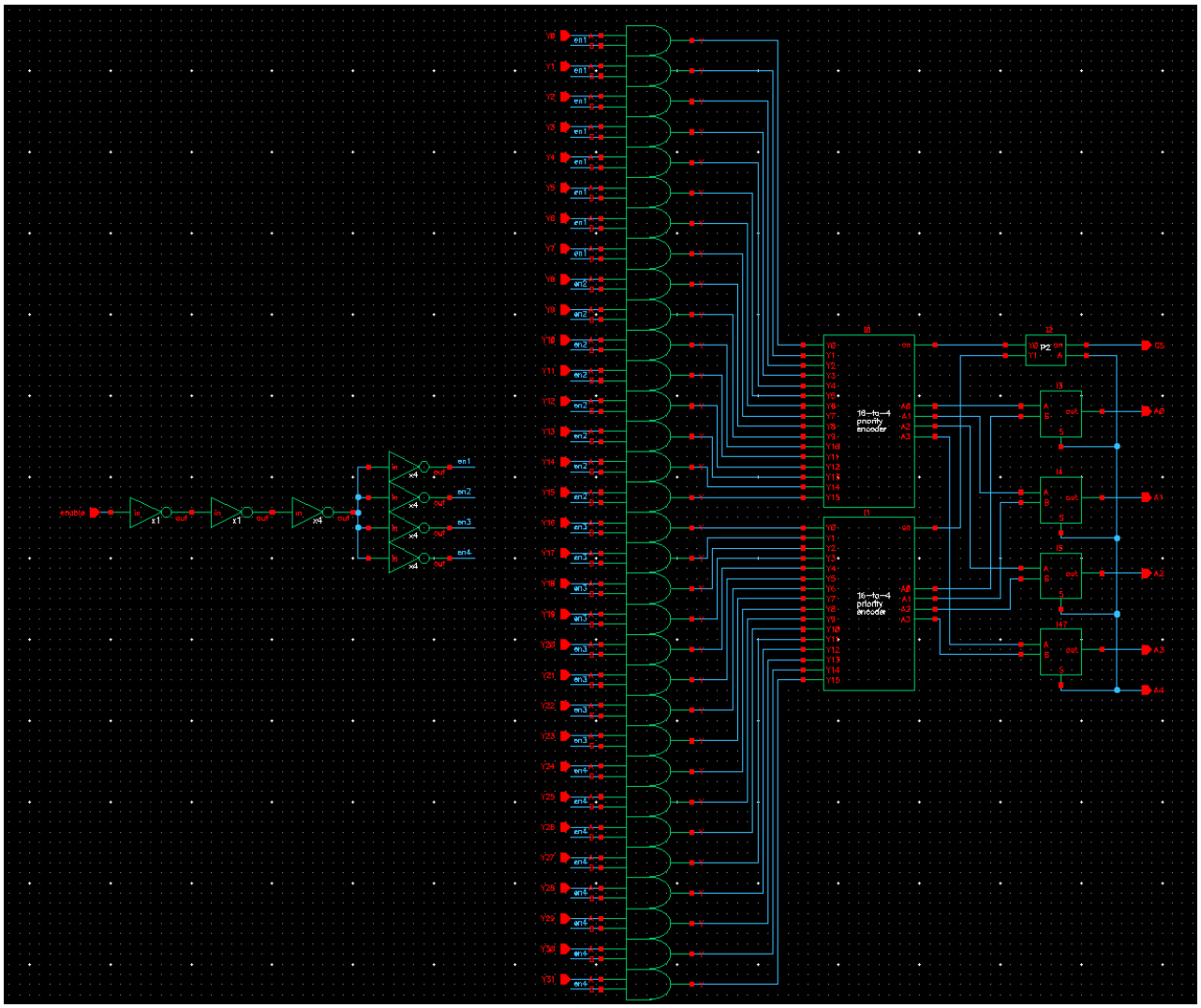
bitcell, row, and full-array operation was also difficult because each phase depended heavily on precise control-signal sequencing. During the design process, our overall approach evolved significantly as the project became more scalable and efficient. The original plan was to implement a smaller 8-bit TCAM architecture to simplify testing and layout complexity. However, after successfully validating the core bitcell and row functionality, we shifted toward a full 32×32 TCAM design to better demonstrate high-performance parallel search capability. This transition required redesigning several parts of the architecture, including larger matchline structures, expanded write circuitry, hierarchical organization, and integration with the priority encoder. The primary limitation on performance came from increased matchline capacitance and extracted parasitic effects within the larger array, which introduced additional delay and dynamic power consumption. As the array size increased, maintaining fast and reliable search operation became more difficult due to longer routing paths and heavier loading on the matchlines. These parasitic effects ultimately constrained the maximum achievable search speed and required careful optimization of routing, buffering, and timing control throughout the design.

Appendix A

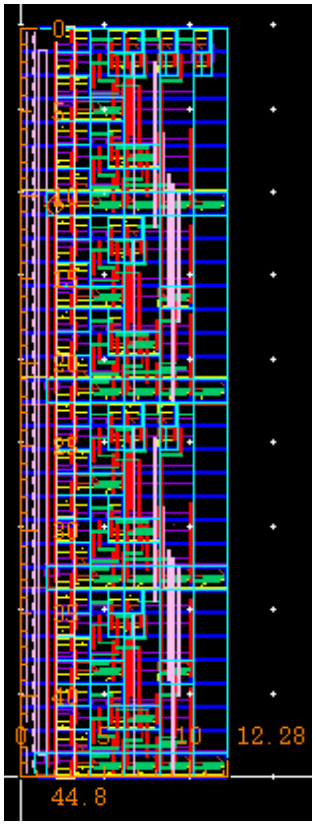
Top Level Layout:



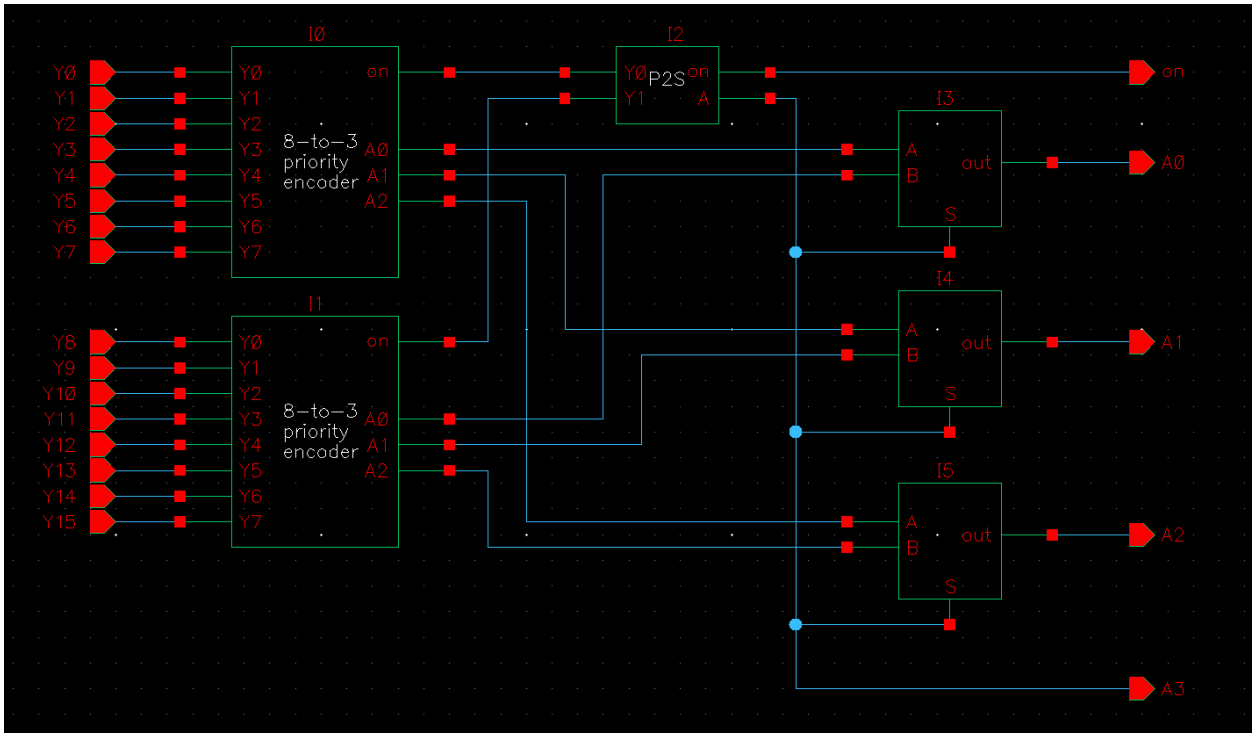
Priority Encoder Schematic:



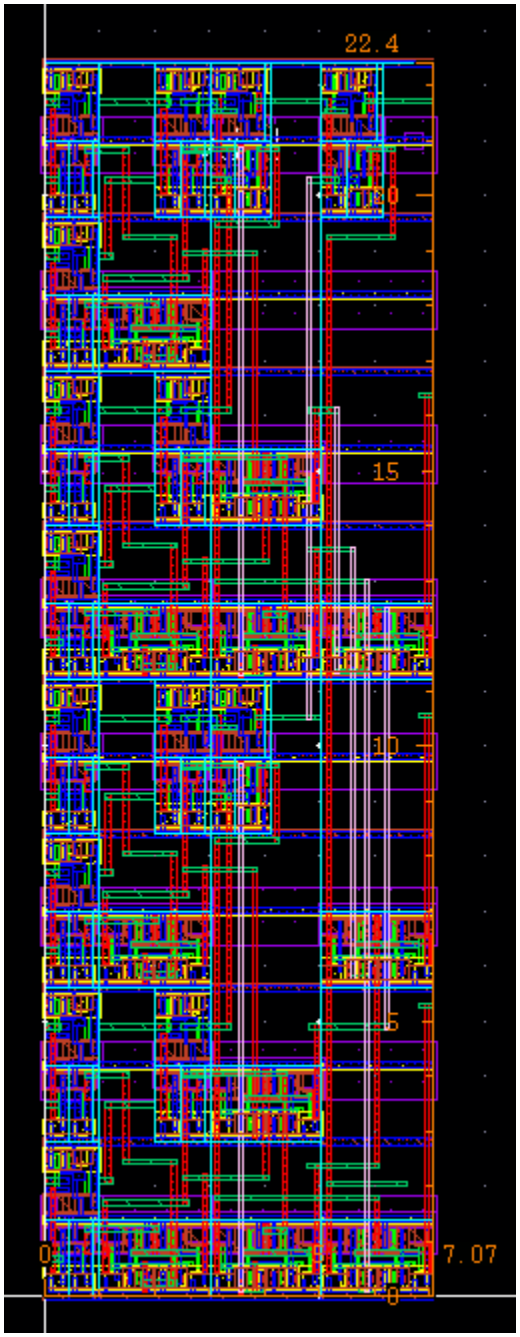
Priority encoder layout:



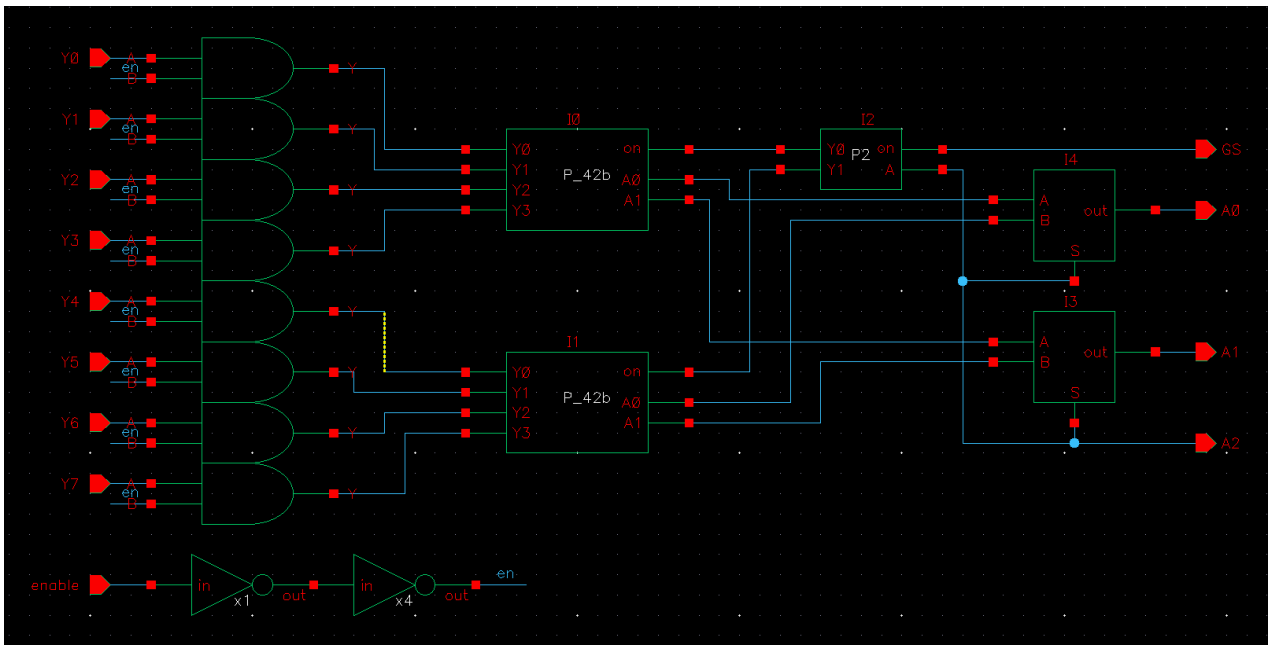
16-to 4 Priority Encoder Schematic:



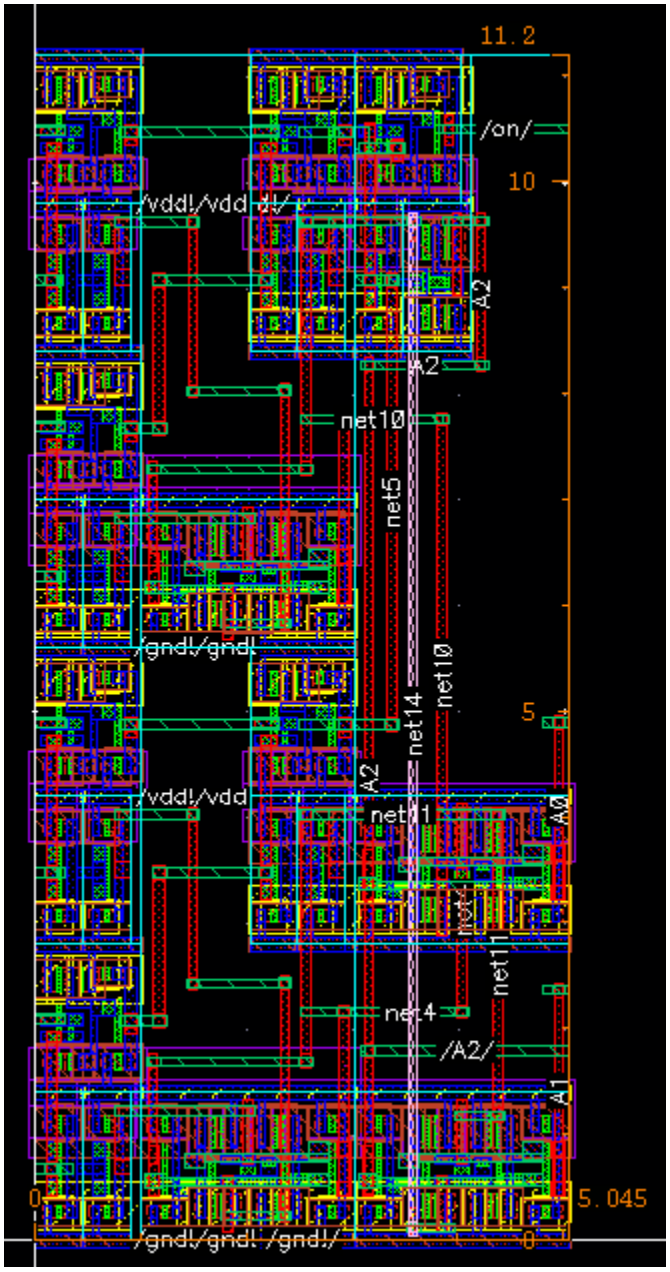
16-to 4 Priority Encoder Layout:



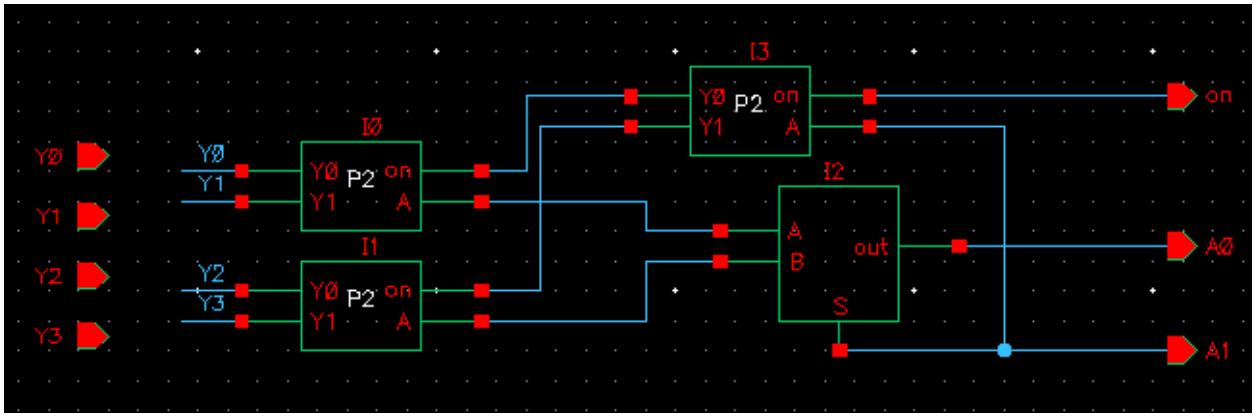
8-to-3 Priority Encoder Schematic:



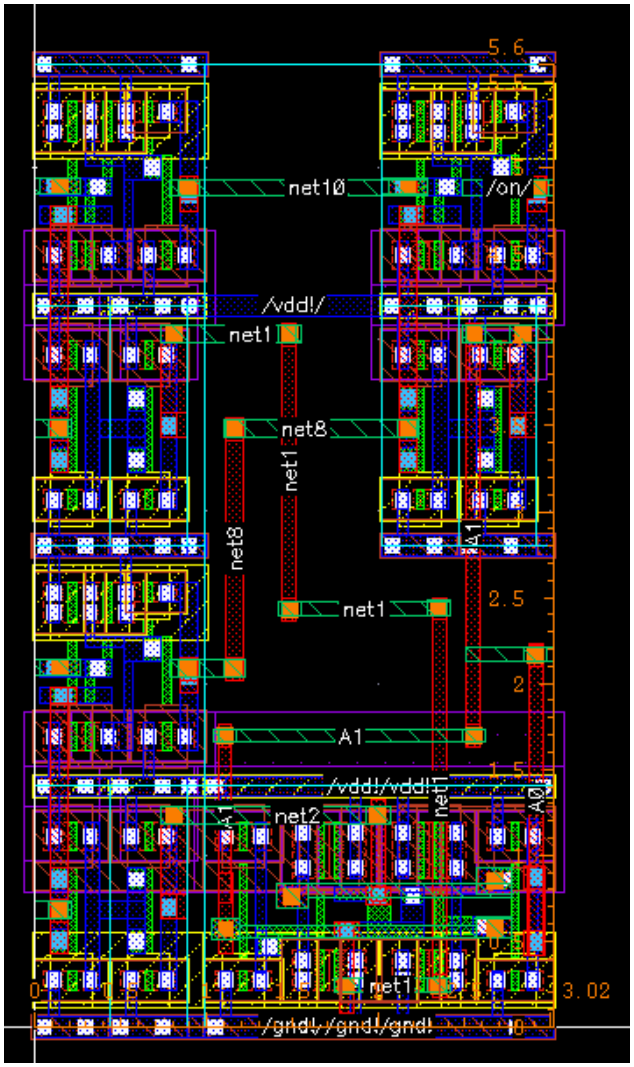
8-to-3 Priority Encoder Layout:



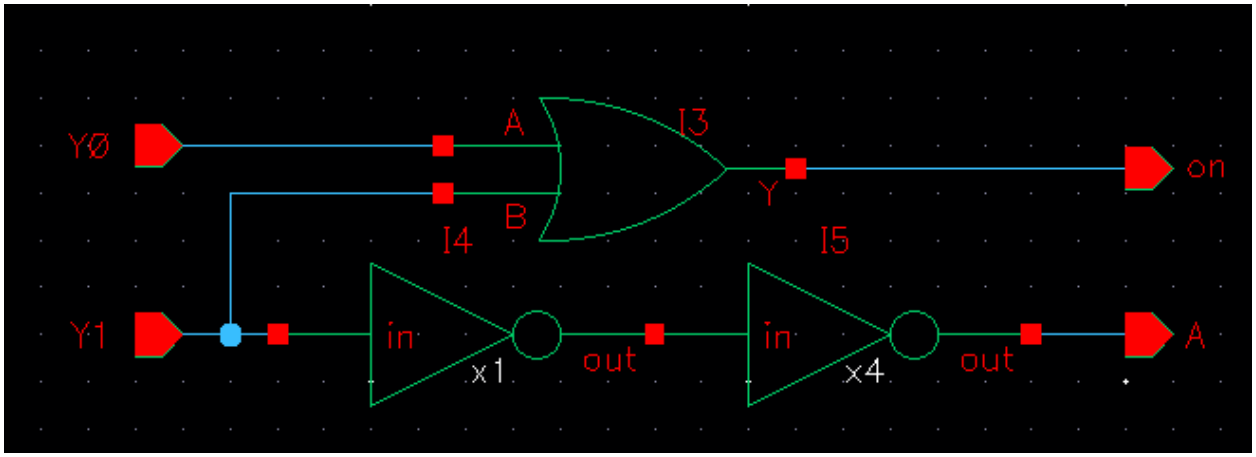
4-to-2 Priority Encoder Schematic:



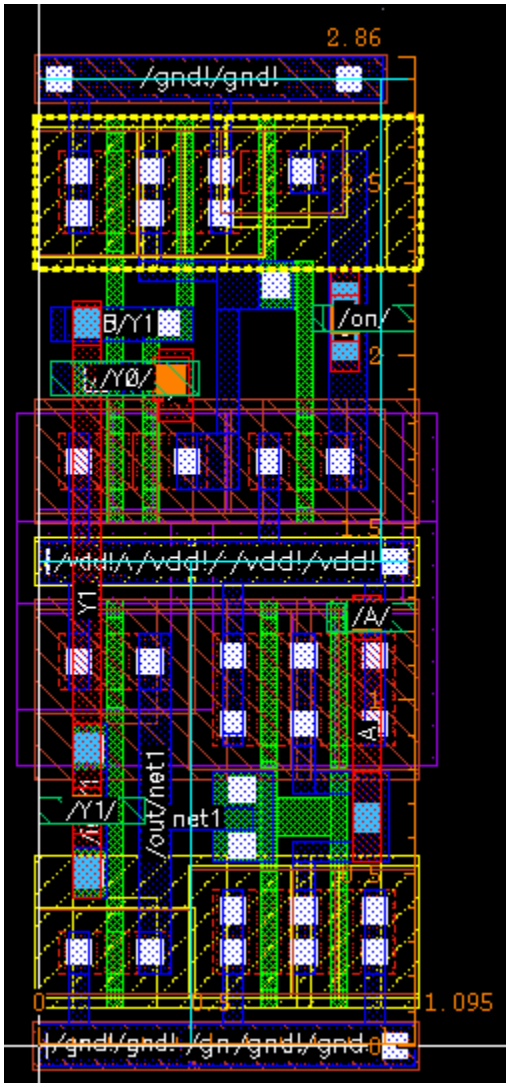
4-to-2 Priority Encoder Layout:



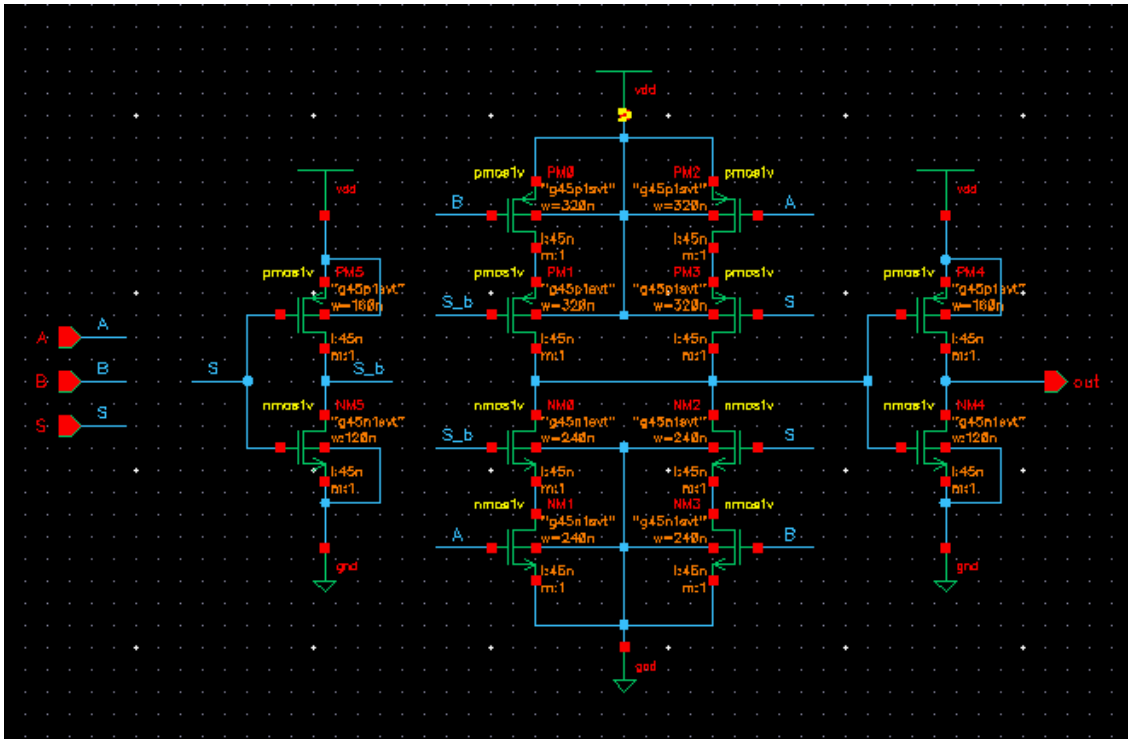
2-to-1 Priority Encoder Schematic:



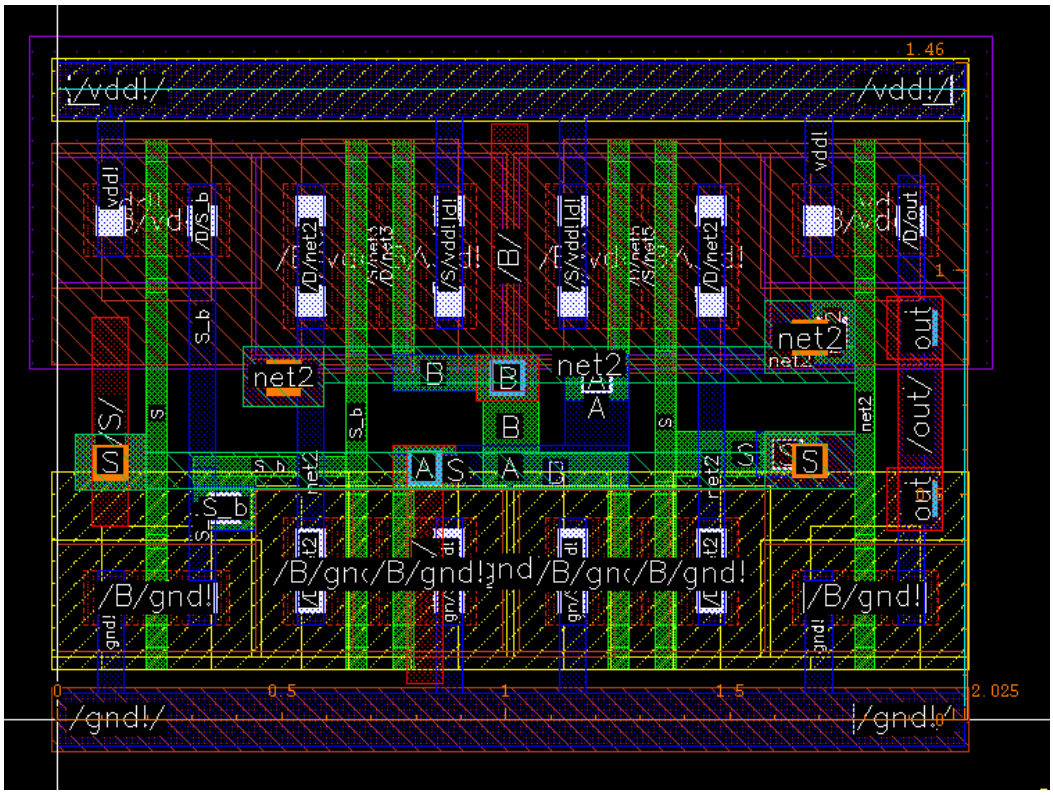
2-to-1 Priority Encoder Layout:



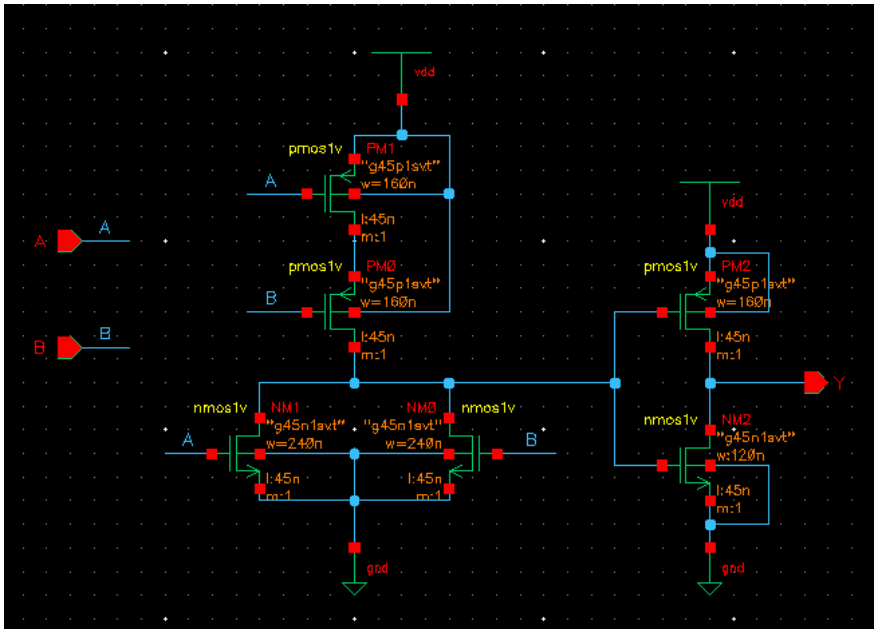
2-to-1 Mux Schematic:



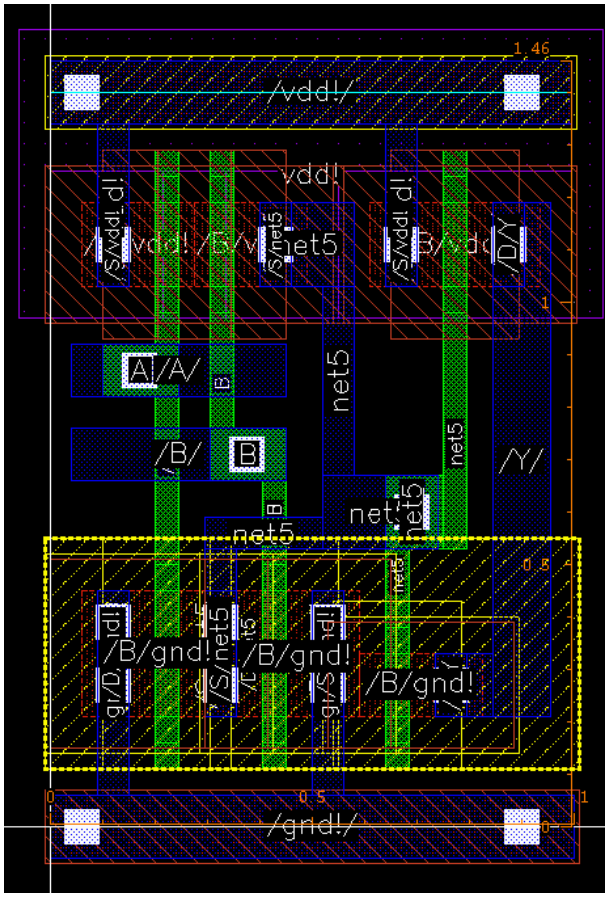
2-to-1 Mux Layout:



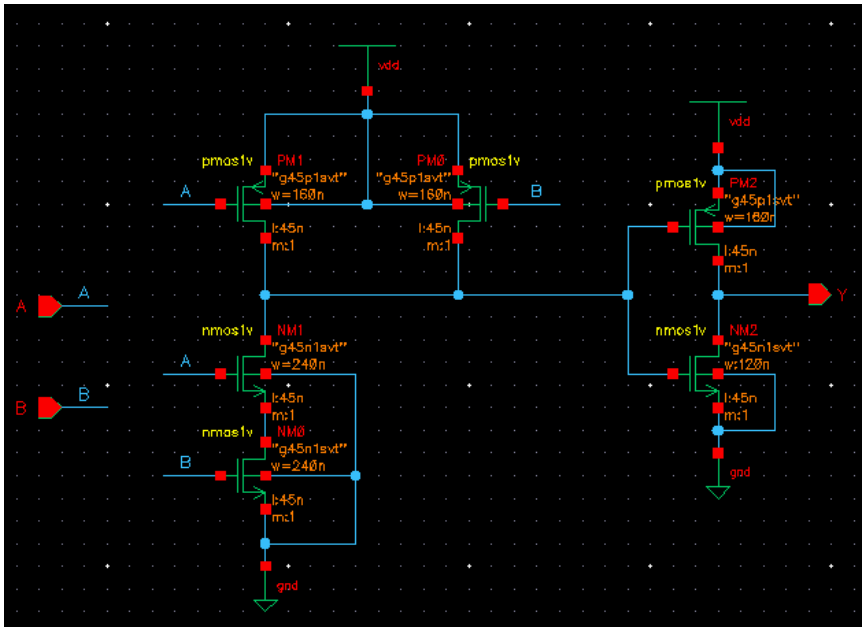
2-Input OR Gate Schematic:



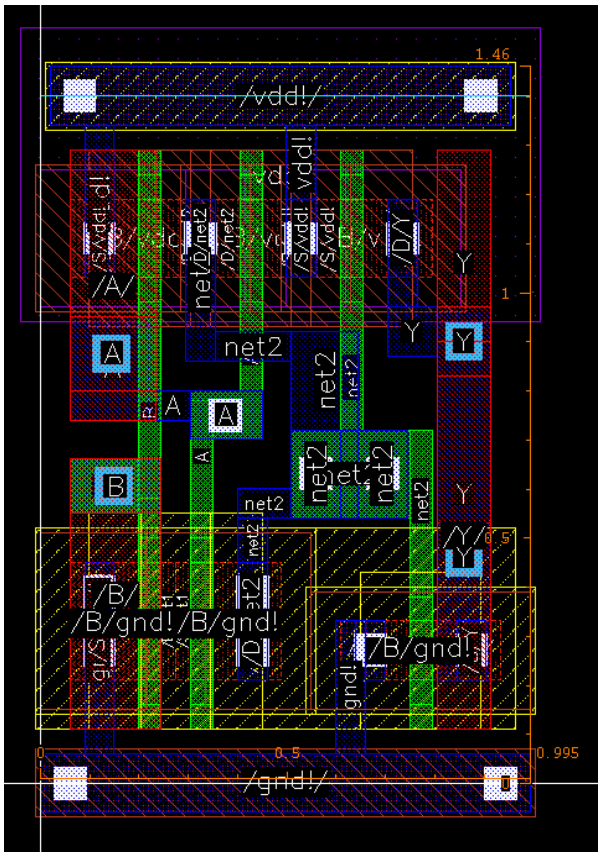
2-Input OR Gate Layout:



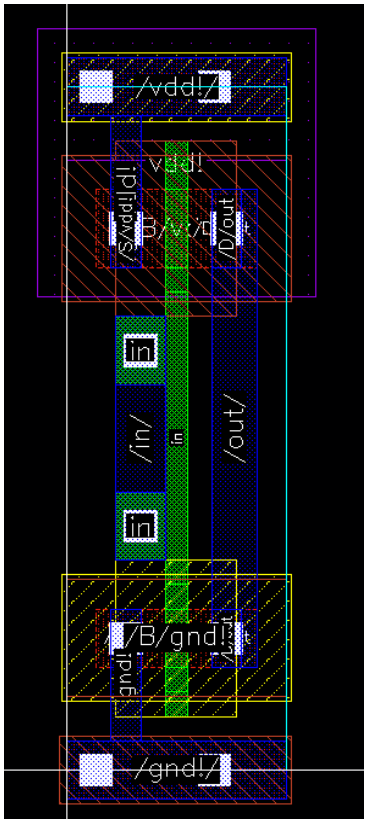
2-Input AND Gate Schematic:



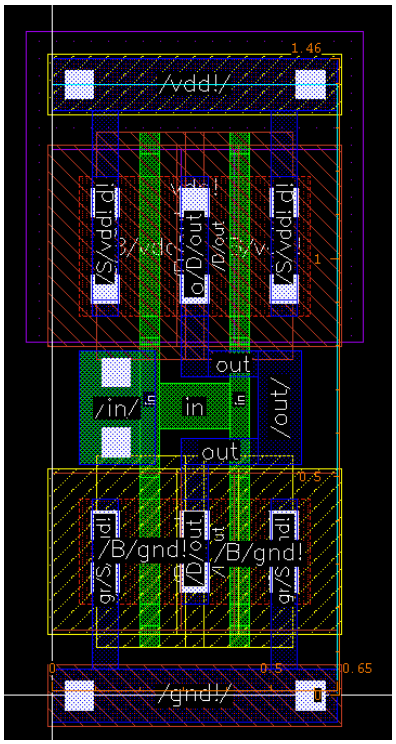
2-Input AND Gate Layout:



1x Inverter Layout:



4x Inverter Layout:



Citations:

[1] S. V. V. Satyanarayana and S. Sriadibhatla, "Design of TCAM Architecture for Low Power and High Performance Applications," Gazi University Journal of Science, vol. 32, no. 1, pp. 164–173, 2019.

[2] K. Pagiamtzis and A. Sheikholeslami, "Content-addressable memory (CAM) circuits and architectures: A tutorial and survey," IEEE Journal of Solid-State Circuits, vol. 41, no. 3, pp. 712–727, Mar. 2006. [Online]. Available: https://ieeexplore.ieee.org/document/5738528

[3] U. Kang et al., "Application exploration for 3-D integrated circuits: TCAM, FIFO, and FFT case studies," IEEE Transactions on Very Large Scale Integration